

ADC

ANUSHA N

ADC INTERFACING WITH 8051

- The analog sensors output data in an analog format which a microcontroller cannot understand. Therefore, to convert this analog data to a digital format, Analog to Digital converters or ADCs are used.
- An analog signal has a continuously changing amplitude with respect to time. A digital signal, on the contrary, is a stream of 0s and 1s.
- An ADC maps analog signals to their binary equivalents. To do this, ADCs use various methods like Flash conversion, slope integration, or successive approximation.

Types of ADC and DAC

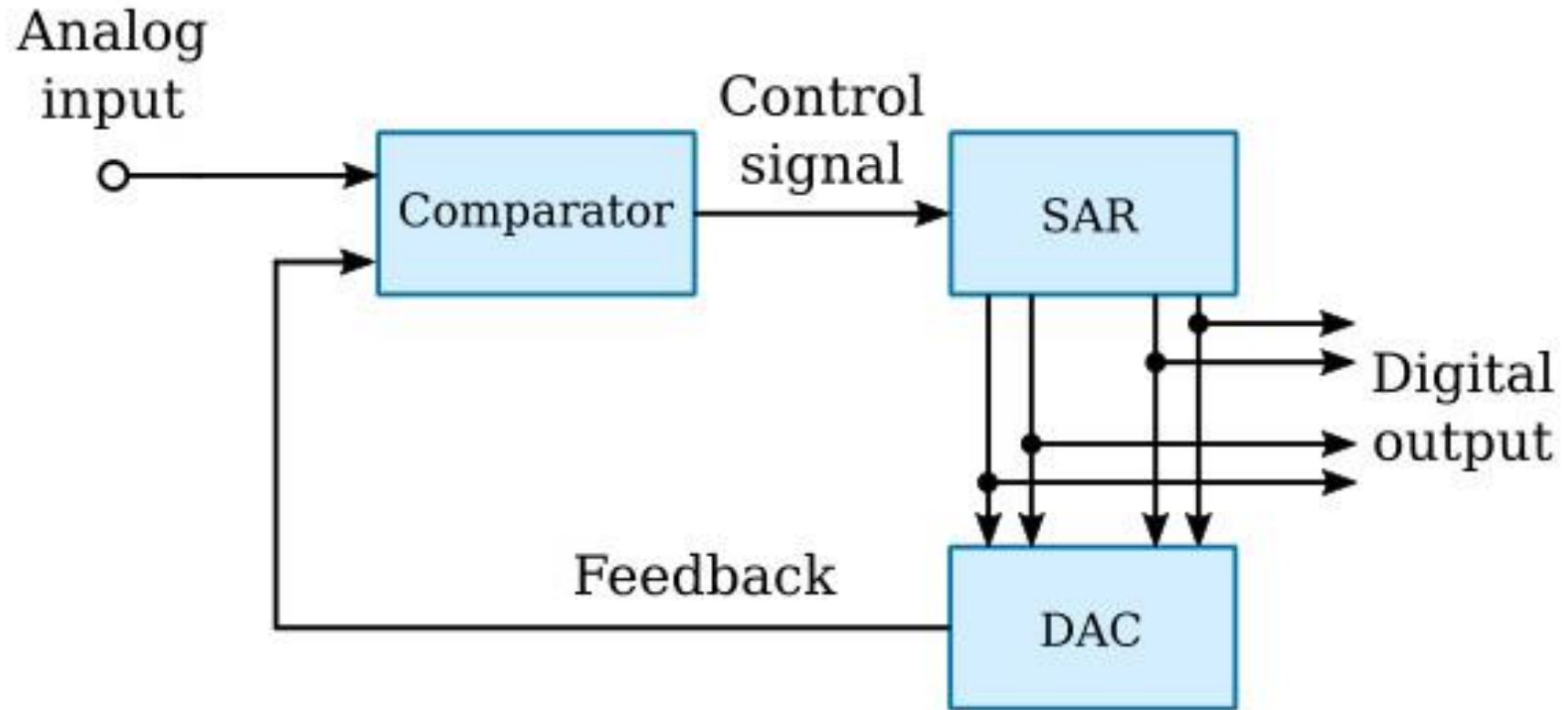
Types of ADC

- Flash ADC
- Successive Approximation ADC
- Single slope ADC

Types of DAC

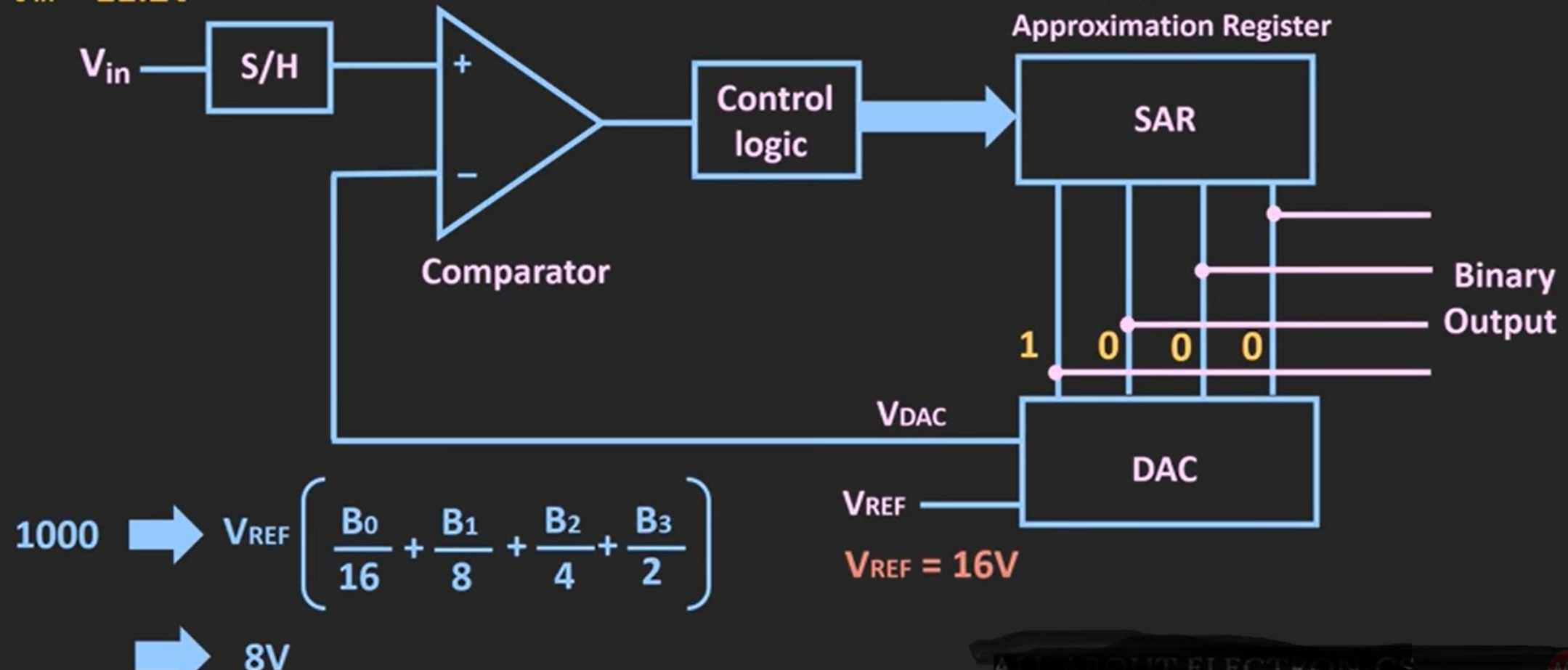
- Voltage divider
- R-2R

Successive Approximation ADC



Successive Approximation ADC

$V_{in} = 11.2V$

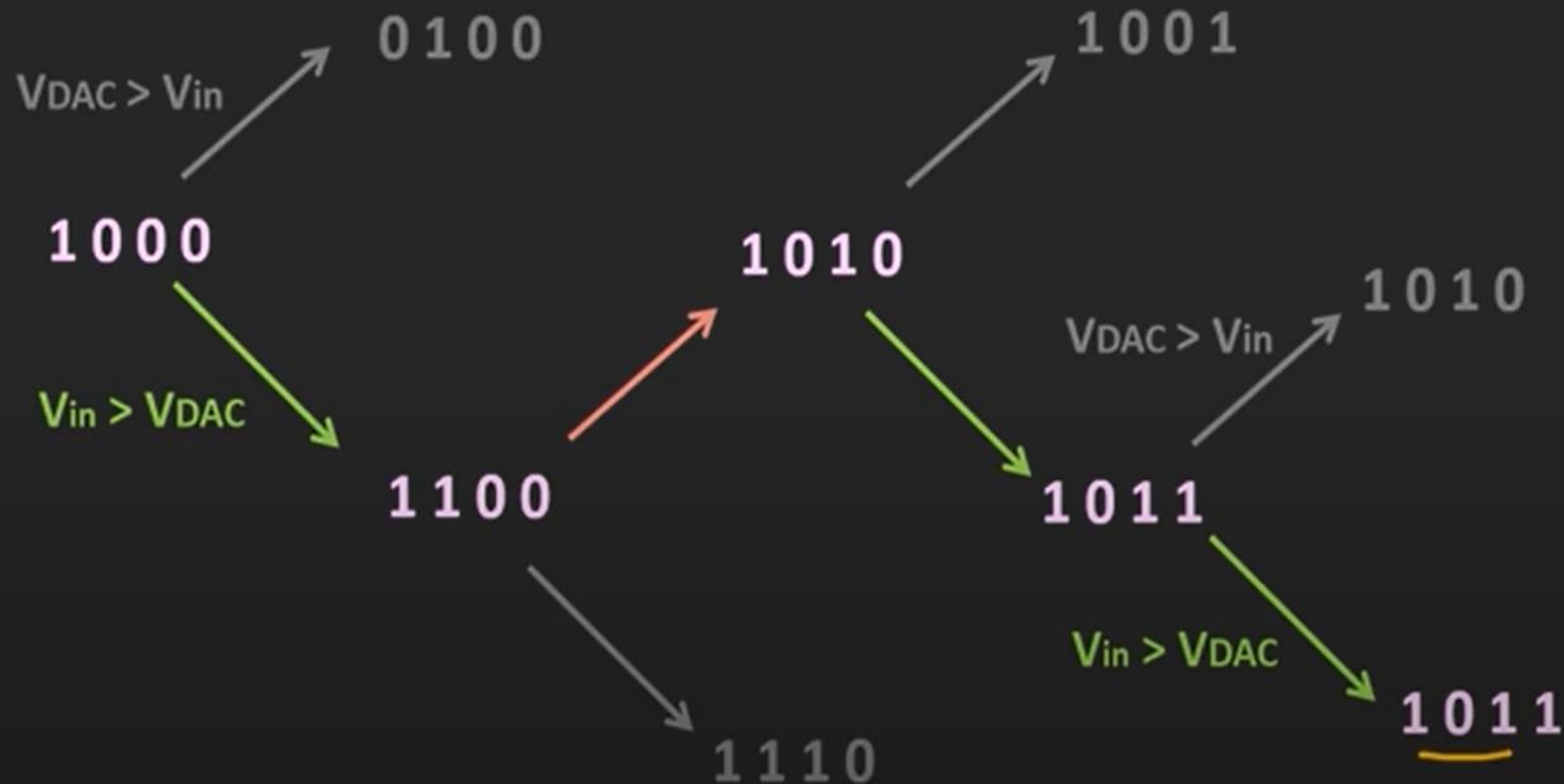


Successive Approximation ADC

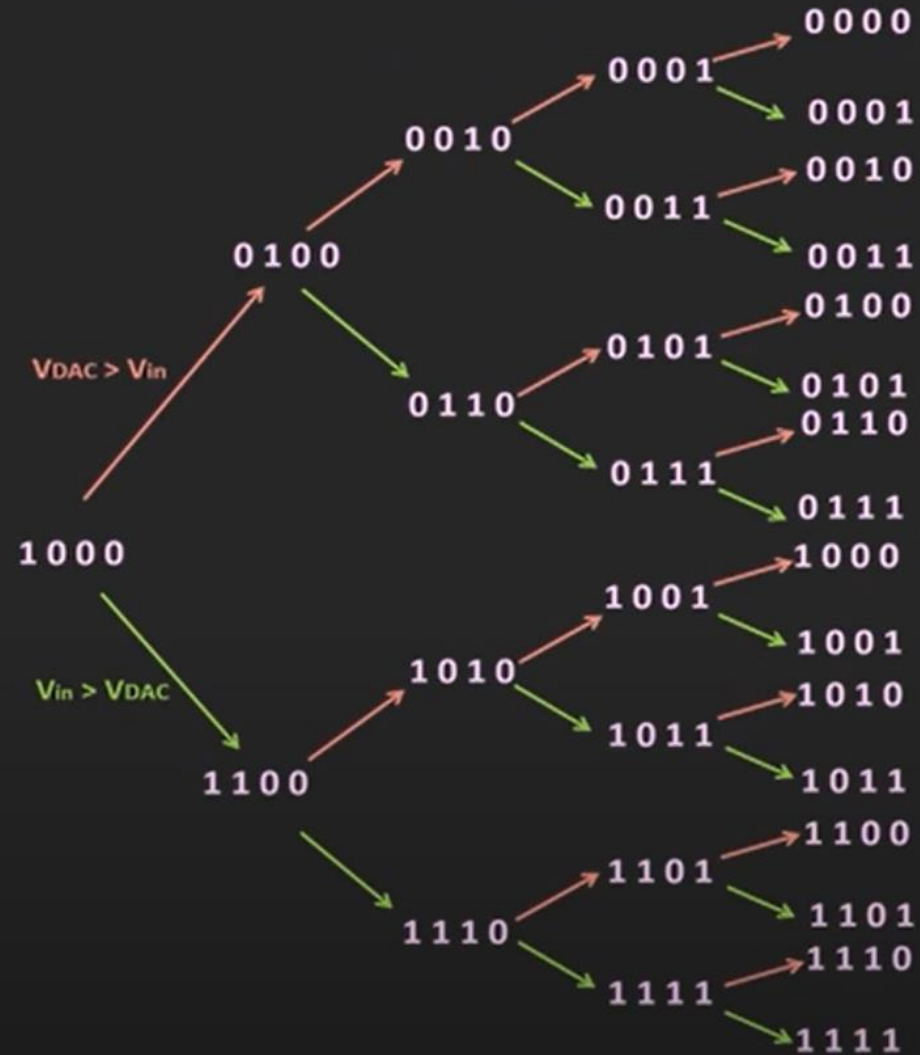
$V_{REF} = 16V$

4-bit ADC

$V_{in} = 11.2V$

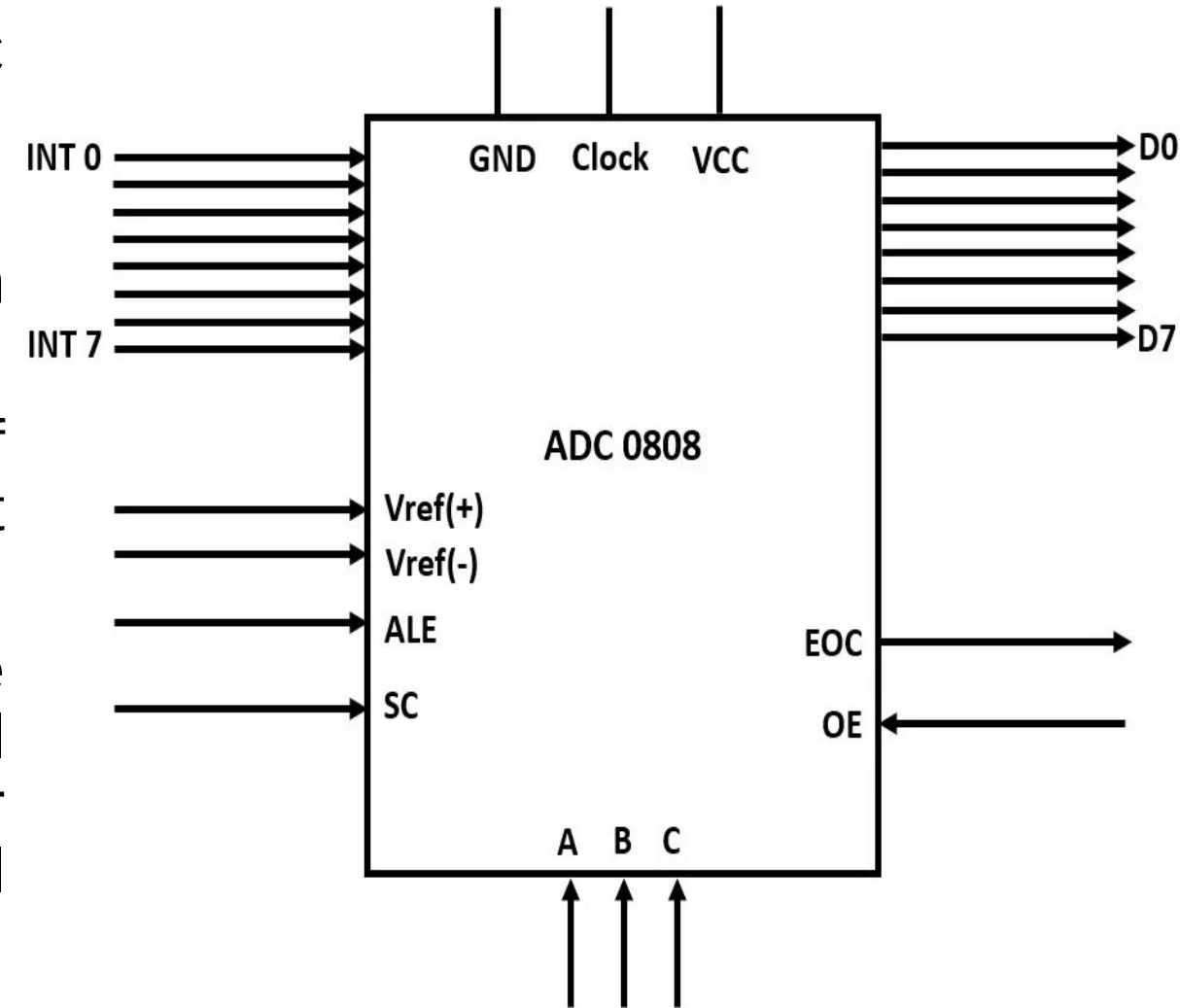


Successive Approximation ADC



ADC 0808

- The ADC 0808 is a popular 8-bit ADC with a step size of 19.53 millivolts.
- It does not have an internal clock. Therefore, it requires a clock signal from an external source.
- It has eight input pins, but only one of them can be selected at a time because it has eight digital output pins.
- It uses the principle of successive approximation for calculating digital values, which is very accurate for performing 8-bit analog to digital conversions.

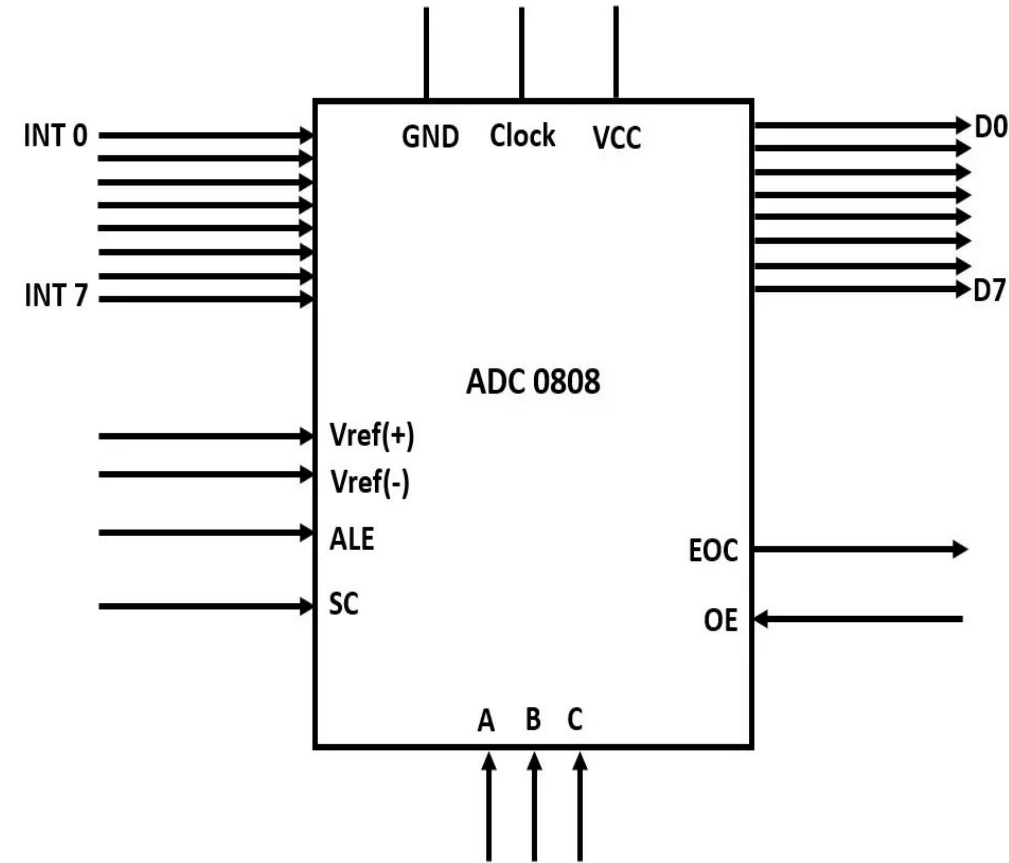


Input pins (INT0-INT7)

- The ADC 0808 has eight input analog pins. These pins are multiplexed together, and only one of them can be selected using three select lines.

Select lines and ALE

- It has three select lines, namely A, B, and C, that are used to select the desired input lines. The ALE pin also needs to be activated by a low to high pulse to select a particular input.



Output pins (D0-D7)

- The ADC has eight output pins that give the binary equivalent of a given analog value.

VCC and Ground

- These two pins are used to provide the required voltage to power the microcontroller. In most cases, the ADC uses 5V DC to power up.

Clock

- As mentioned earlier, the 0808 does not have an internal clock and needs an external clock signal to operate. It uses a clock frequency of 20Mhz, and using this clock frequency it can perform one conversion in 100 microseconds.

VREF (+) and VREF (-)

- These two pins are used to provide the upper and the lower limit of voltages which determine the step size for the conversion. Here Vref(+) has a higher voltage, and Vref(-) has the lower voltage. If Vref(+) has an input voltage 5v and Vref(-) has a voltage of 0v then the step size will be $5v - 0v / 2^8 = 15.53 \text{ mv}$.

Start conversion

- This pin is used to tell the ADC to start the conversion. When the ADC receives a low to high pulse on this pin, it starts converting the analog voltage on the selected pin to its 8-bit digital equivalent.

End of conversion

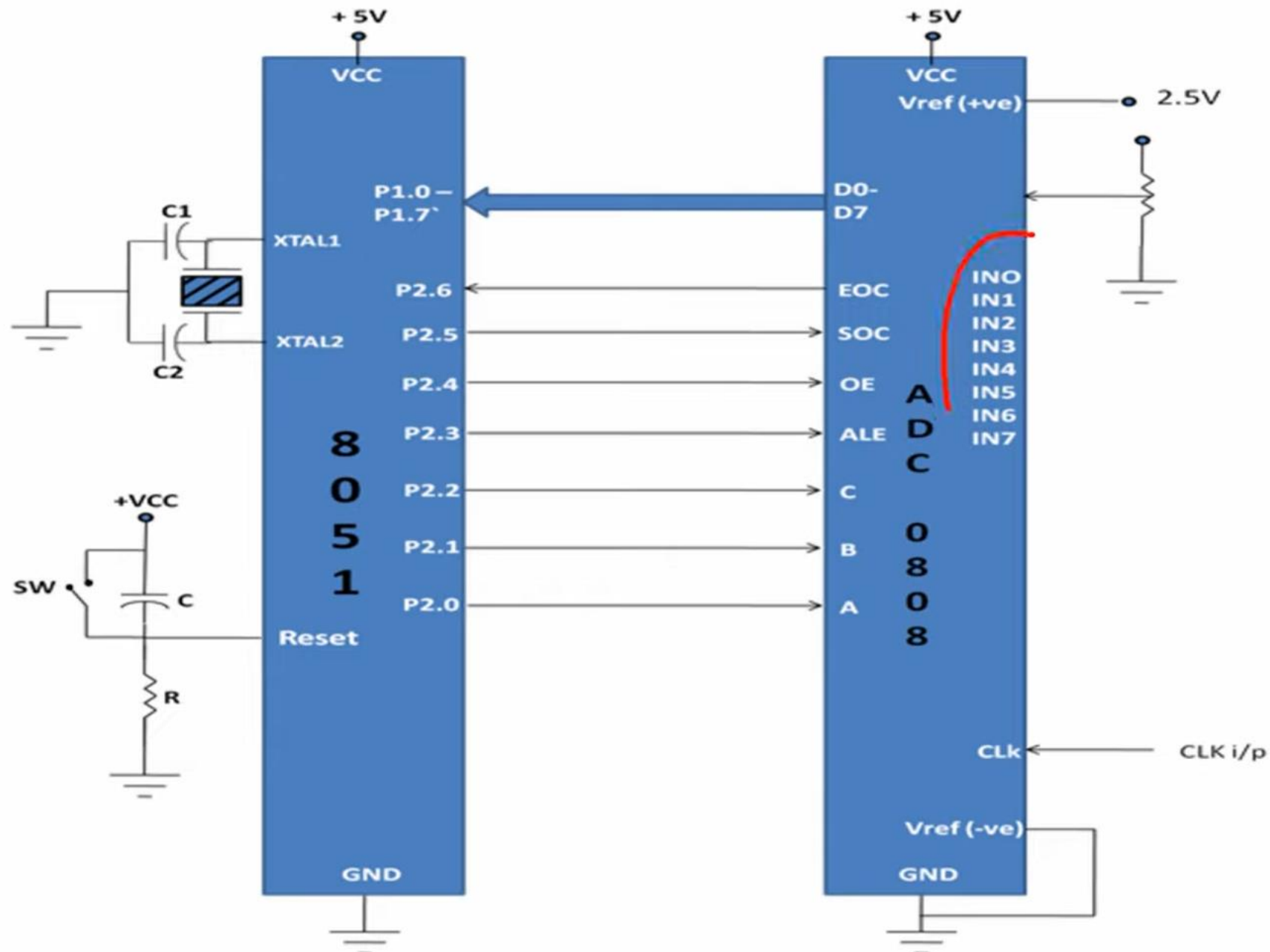
- Once the conversion is complete, the ADC sends low to high signal to tell a microcontroller that the conversion is complete and that it can extract the data from the 8 data pins.

Output enable

- This pin is used to extract the data from the ADC. A microcontroller sends a low to high pulse to the ADC to extract the data from its data buffers

- The ADC 0808/0809 is the 8-bit analog to digital converter.
- It has 8-channels IN0-IN7.
- It has three address lines i.e. Pin A,B,C and ALE are used to select one of the analog channel of total 8 channels as shown below:

A	B	C	Channel
0	0	0	IN0
0	0	1	IN1
0	1	0	IN2
0	1	1	IN3
1	0	0	IN4
1	0	1	IN5
1	1	0	IN6
1	1	1	IN7



1. Apply the analog input to one of the input channel.
2. Select the respective channel
3. Send output High pulse on ALE
4. Send output High pulse on SoC
5. Wait for low pulse on EoC
6. Make OE high to enable microcontroller to read output data
7. Read the digital output on port 1
8. Repeat steps 3 to 7 for next conversion.

```

SOC EQU P2.5
EOC EQU P2.6
OE EQU P2.4
ALE EQU P2.3
ADDR_C EQU P2.2
ADDR_B EQU P2.1
ADDR_A EQU P2.0
ORG 0000H
MOV P1,#0FFH
SETB P2.5
CLR SOC
CLR OE
CLR ALE

```

```

AGAIN: CLR ADDR_C
CLR ADDR_B
CLR ADDR_B
SETB SOC
SETB ALE
ACALL DELAY
CLR SOC
CLR ALE
HERE: JB EOC HERE
SETB OE
MOV A,P1
AGAIN: SJMP AGAIN

```